

Sistema de alertas operacionales

Caso técnico Rappi · Data-driven + motor experto + agente Telegram

Demo técnica

21 de marzo de 2026

Agenda

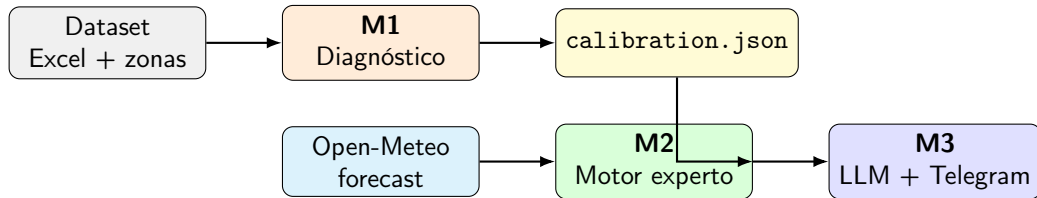
- 1 Visión general
- 2 Módulo 1 — Diagnóstico operacional
- 3 Módulo 2 — Motor de alertas
- 4 Módulo 3 — Agente Telegram
- 5 Dashboard web (Django)
- 6 Demo en vivo y cierre

- **Contexto:** operación de delivery sensible al clima (lluvia, saturación de repartidores).
- **Objetivo:** avisar con **anticipación** (ventana corta) y **números accionables** (zonas, riesgo, incentivos en MXN).
- **Enfoque:** *data-driven first* — lo que decide no es “opinión del modelo”, sino reglas y coeficientes **calibrados con histórico**.

Los tres módulos en una frase

- Módulo 1 Diagnóstico:** del Excel y el notebook salen sensibilidades, umbrales y coeficientes por zona.
- Módulo 2 Motor:** pronóstico en vivo (Open-Meteo) + reglas IF-THEN + salida estructurada.
- Módulo 3 Agente:** el mismo contexto en JSON → lenguaje natural (LLM o plantilla) → Telegram.

Arquitectura del pipeline



El M3 reutiliza el **mismo motor** que el M2; añade formato operativo y canal.

- **Entrada:** panel histórico (`data/rappi_delivery_case_data.xlsx`): órdenes, earnings, precipitación, hora, zona, saturación, etc.
- **Herramienta principal:** notebook `01_diagnostico_operacional.ipynb`.
- **Salida conceptual:** entender **ratio**, saturación y relación precipitación–operación **por zona**.
- **Salida para ingeniería:** parámetros que alimentan `calibration.json` (vía export o notebook alineado).

Módulo 1: qué se estima por zona

- `precip_coef` — efecto marginal de la precipitación sobre el ratio (regresión tipo notebook).
- `sensitivity_index` — qué tan “reactive” es la zona respecto al resto (normalización de coeficientes).
- `alert_precip_mm_hr` — umbral de lluvia a partir del cual tiene sentido alertar (p. ej. percentil en condiciones de saturación).
- `mm_precip_healthy_to_saturation_linear` — orden de magnitud mm/h en el tramo saludable → saturación.
- `base_earnings_mxn` — referencia de incentivo base para calcular `earnings_from` → `earnings_to`.

Cinco hallazgos cuantificados (patrones $\rightarrow$ M2)

- 1 **Panel:** $30 \times 24 \times 14 = 10\,080$ filas; métrica `ratio` = `ORDERS/CONNECTED_RT`; saturación si $> 1,8$.
- 2 **Precip vs ratio (con controles):** correlación $\approx 0,32$; OLS `ratio` $\sim$ `precip` + `C(HOUR)` + `C(ZONE)` $\Rightarrow$ Adj. $R^2 \approx 0,72$ (notebook P2).
- 3 **Condicional:** $\mathbb{P}(\text{sat} \mid \text{precip} > 0,5) \approx 0,31$ vs $\approx 0,04$ si `precip` $\leq 0,5$ mm/h ($\sim 8\times$).
- 4 **Por zona:** sensibilidad y mm para pasar tramo saludable $\rightarrow$ saturación difieren (ej. Santiago vs Mitras en `calibration.json`).
- 5 **Datos/geo:** 12/14 WKT válidos; motor usa centroides como respaldo. Umbrales `alert_precip_mm_hr` exportados por zona.

Documento: `modulo1_diagnostico/HALLAZGOS_M1.md`

- Informe LaTeX/PDF en `modulo1_diagnostico/Diagnóstico Operacional/`.
- Figuras generadas por el notebook en `modulo1_diagnostico/figures/`.
- Script `export_calibration_from_m1.py` (M2) reproduce la lógica del notebook para regenerar `calibration.json` sin abrir Jupyter.

Puente M1 → M2

Excel + notebook $\Rightarrow$ conocimiento cuantitativo por zona



calibration.json (umbrales + coeficientes + sensibilidad)

- **API:** Open-Meteo (sin API key) — pronóstico horario de precipitación.
- **Unidades:** mm acumulados por hora $\approx$ intensidad mm/h (alineado al dataset).
- **Zona horaria:** America/Monterrey en el cliente.
- **Horizonte por defecto:** 2 horas (balance precisión vs tiempo de reacción).

- Polígonos WKT por zona en el Excel; **Shapely** para geometría.
- Consulta al clima en un punto **representativo dentro del polígono** (no solo centroide arbitrario cuando hay WKT válido).
- Si el WKT falla, **respaldo** con coordenadas de ZONE_INFO.
- Opcional: agregación por rejilla (geo_pipeline) para escenarios más finos.

- ➊ Para cada zona: máxima precipitación prevista en la ventana vs `alert_precip_mm_hr`.
- ➋ Elegir zona **primaria** (mayor exceso sobre umbral).
- ➌ Proyección de **ratio** con `precip_coef` (techo/piso razonables).
- ➍ Clasificación de **riesgo**: BAJO, MEDIO, ALTO, CRITICO.
- ➎ **Incentivo numérico**: multiplicador acotado que sube con riesgo, sensibilidad y exceso de lluvia.
- ➏ **Zonas secundarias** a vigilar (top sensibilidad).

Módulo 2: anti alert fatigue (debounce)

- Estado en `.alert_state.json` por zona.
- No reenvía la **misma o menor severidad** dentro de un **TTL** (por defecto ~ 45 min).
- **Sí reenvía** si hay **escalada** (p. ej. MEDIO $\rightarrow$ CRITICO).
- TTL configurable: variable `ALERT_DEBOUNCE_TTL_SEC` en `caso_tecnico/.env`.

`modulo2_motor_alertas/run_alert_engine.py`

- `python run_alert_engine.py` — pronóstico real, salida por consola.
- `python run_alert_engine.py -demo` — simula lluvia fuerte en Santiago sin red.
- `python run_alert_engine.py -recalibrate` — regenera `calibration.json` desde el Excel y luego ejecuta.

Documento de reglas: `latex/motor_reglas.tex` → PDF de una página.

- Mismo flujo meteorológico + **mismo motor M2**.
- El contexto experto va a **RAG-lite**: el “retrieval” es el **JSON estructurado** del motor, no documentos arbitrarios.
- **LLM** (Ollama / OpenAI) genera JSON con esquema fijo; con **OpenAI**, orquestación con **LangChain** (LCEL) por defecto; validación y fallback a **plantilla determinista**.
- Envío con **python-telegram-bot** al canal configurado.

Módulo 3: mensaje para el jefe de operaciones (~ 10 s)

El texto busca incluir:

- Zona y **nivel de riesgo**
- Qué se espera en la ventana (lluvia, horizonte)
- Paralelo **histórico** (frase guía desde el pipeline)
- Acción: **earnings X $\rightarrow$ Y MXN y minutos**
- **Zonas secundarias** a monitorear

- `caso_tecnico/.env`: `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`.
- Bot como **administrador** del canal con permiso de publicar.
- `LLM_PROVIDER`: `ollama` | `openai` | `auto`.
- **Nunca** commitear tokens; revocar en BotFather si hay fuga.

Módulo 3: comandos útiles para la demo

Desde la raíz caso_tecnico/:

Pruebas

```
python modulo3_agente_telegram/run_agent.py -test-telegram  
python modulo3_agente_telegram/run_agent.py -demo -dry-run  
python modulo3_agente_telegram/run_agent.py -demo -validate
```

Canal en vivo

```
python modulo3_agente_telegram/run_agent.py -demo -force-send
```

- **Debounce visible:** si se suprime la alerta por TTL, se envía un **aviso corto** a Telegram (salvo `-dry-run`).
- **Registro:** `.alert_events.jsonl` por cada alerta enviada.
- **Resumen diario:** `-daily-summary` (lista de eventos + contraste con archivo Open-Meteo del día).
- Cron nocturno posible para el resumen (ver README del módulo 3).

Módulo 3: monitor continuo (monitor_loop.py)

- **Bucle operativo:** cada N segundos vuelve a llamar a Open-Meteo, ejecuta el **mismo motor M2** y aplica debounce antes de Telegram.
- **Orquestación:** cadena **LangChain LCEL** (timestamp $\rightarrow$ run_operational_tick $\rightarrow$ resumen); no sustituye las reglas del experto.
- **Intervalo por defecto: 600 s** (10 min), vía `MONITOR_INTERVAL_SEC` en `.env` o flag `-interval-sec`. Mínimo aplicado en código: **60 s**.
- **Auditoría / UI:** cada ciclo se añade a `modulo2_motor_alertas/.monitor_ticks.jsonl` y se actualiza `.monitor_status.json` (lo lee el dashboard).

```
python modulo3_agente_telegram/monitor_loop.py | -once -demo -dry-run (prueba)
```

Monitor vs debounce (dos relojes distintos)

- **Monitor** = cada cuánto se **consulta** el clima y se reevalúa si *podría* haber alerta (típ. cada 10 min).
- **Debounce** = cada cuánto se permite **repetir** la misma severidad en la misma zona por Telegram (TTL ~45 min, ALERT_DEBOUNCE_TTL_SEC).
- **Escalada** MEDIO→CRITICO (y similares) **sí** dispara envío aunque no haya pasado el TTL completo.

- Interfaz local para **explorar el Excel** (preview, describe, ZONE_INFO), gráficos por zona y ratio por hora.
- Vista **Pipeline** (diagrama $M1 \rightarrow M2 \rightarrow M3$) y **Calibración** (calibration.json formateado).
- Galería de **figuras del notebook** (modulo1_diagnostico/figures/).
- Ruta /monitor/: último ciclo del monitor_loop, historial de ticks y últimas alertas en .alert_events.jsonl (refresco automático vía /api/monitor.json).
- Desde inicio: botones para **forzar envío** a Telegram (run_agent.py -force-send) — solo en entorno de confianza.

```
cd django_viz && python manage.py runserver 8080 ⇒ /monitor/
```

Sugerencia de guion (8–12 minutos)

- 1 Arquitectura (esta presentación o diagrama del pipeline).
- 2 **M1/M2:** abrir `calibration.json` (umbral + sensibilidad de una zona).
- 3 `cd modulo2_motor_alertas && python run_alert_engine.py -demo.`
- 4 **M3:** `run_agent.py -demo -dry-run` → `-force-send` o canal real (`-test-telegram` primero si hace falta).
- 5 **Monitor:** `monitor_loop.py -once -demo -dry-run`; en otra terminal, `dashboard /monitor/` con `runserver`.
- 6 Opcional: `-daily-summary -dry-run` si ya hay líneas en `.alert_events.jsonl`.

Checklist de entregables (repo)

- **M1:** notebook + figuras + informe LaTeX en modulo1_diagnostico/.
- **M2:** decision_engine.py, debounce.py, cliente Open-Meteo, calibration.json, run_alert_engine.py.
- **M3:** rag_chain.py, Telegram, run_agent.py, monitor LangChain, registro JSONL.
- **Ops:** docs/criterios_evaluacion_demo.md, modulo3_agente_telegram/docs/qa_arquitectura.md, docs/retroalimentacion_y_escalado.md, .env.example, requirements.txt.
- **Demo:** esta presentación (demo_caso_tecnico/presentacion_demo.tex) + PDF generado.

- **Falso positivo meteorológico:** umbral por zona + segunda lectura operativa (radar); evolución: confirmar con observaciones.
- **Alert fatigue:** debounce + escalada; mensajes cortos y accionables.
- **Escalado a otra ciudad:** nuevo Excel + recalibración + mismos scripts; TELEGRAM_CHAT_ID por canal.

Guía de demo rúbrica: docs/criterios_evaluacion_demo.md. Q&A:
modulo3_agente_telegram/docs/qa_arquitectura.md, docs/retroalimentacion_y_escalado.md.

Gracias

Preguntas